DEVELOPMENT WORKFLOW

Wave — Workflow de desarrollo eficiente

Flujo de trabajo y toolkit de scripts para no perder cambios y reducir errores en wave-app

Proyecto:	wave-app (github.com/Danielbarrantesv/wave-app)
Especificación:	Wave v2.4.0 (com.mp3tify, versionCode 6)
Sandbox base:	Wave2 (/home/z/my-project)
Autor:	Daniel Barrantes Villarevia
Fecha:	6 de noviembre de 2026
Versión doc:	1.0

Índice

Para actualizar los números de página tras editar el documento, haga clic derecho sobre el índice y elija «Actualizar campos».

1. Resumen ejecutivo.....

1. Resumen ejecutivo.....1

2. Diagnóstico.....2

2.1 Por qué se pierden cambios.....2

2.2 Por qué se acumulan errores.....2

3. Principios del nuevo flujo.....3

3.1 GitHub es la única fuente de verdad.....3

3.2 Build local antes de push, sin excepciones.....3

3.3 Toolchain reproducible y versionado.....3

3.4 Sincronización frecuente vs. publicación cuidadosa.....4

3.5 Recuperación ante desastre como ciudadano de primera clase.....4

4. Setup inicial (bootstrap).....4

4.1 Pasos del setup.....4

4.2 Verificación.....5

5. Flujo de sesión diaria.....6

5.1 Las cinco etapas.....6

5.2 Detalle de cada etapa.....6

5.3 Diagrama de decisión.....7

6. Catálogo de scripts.....7

6.1 Tabla maestra.....8

6.2 Variables de entorno configurables.....8

7. Git hooks y verificación pre-push.....8

7.1 Qué hace el hook exactamente.....9

7.2 Cuándo saltarse el hook (emergencias).....9

7.3 Instalación y desinstalación.....9

8. Protocolo de recuperación ante desastre....10
8.1 Qué hace recover.sh paso a paso....10
8.2 Dry-run y modos alternativos....10
9. Anti-patrones a evitar....10
9.1 Editar directo en GitHub web....11
9.2 Usar sync.sh para publicar versiones estables....11
9.3 git push --force a main sin aviso....11
9.4 WAVE_SKIP_BUILD_CHECK=1 como costumbre....11
9.5 Acumular horas de trabajo sin commitear....12
9.6 Reinstalar el toolchain de cero sin consultar el respaldo....12
10. Mapeo con reglas existentes de la spec Wave....12
Apéndice A — Comandos rápidos....13
A.1 Inicio de sesión....13
A.2 Trabajo....13
A.3 Sincronización y publicación....13
A.4 Emergencias....13
Apéndice B — Estructura de archivos del toolkit....13

1. Resumen ejecutivo

El proyecto Wave — reproductor de música offline para Android escrito en Kotlin con Jetpack Compose — se desarrolla en un sandbox que se recicla entre sesiones y se publica en GitHub Actions. La especificación técnica v2.4.0 ya define reglas de operación (capítulo 13.1) y reglas de oro (capítulo 15.1) que apuntan exactamente a los dos problemas reportados: no perder cambios y no acumular errores. Sin embargo, en la práctica esas reglas se incumplen porque faltan herramientas concretas que las automaticen.

Este documento entrega un toolkit de siete scripts bash que operan desde el sandbox Wave2 y materializan cada regla de la especificación. El flujo propuesto consta de cinco etapas: bootstrap del toolchain, inicio de sesión con clone/pull y activación de entorno, edición del código, sincronización rápida para no perder cambios, y publicación confiable con verificación de build antes del push. Adicionalmente se instalan git hooks que bloquean cualquier push a la rama main si el build local falla, eliminando la causa más frecuente de errores en GitHub Actions.

> **Resultado esperado:** cero cambios perdidos por reciclaje del sandbox, cero pushes que rompan el pipeline de Actions, y un mecanismo de recuperación ante desastre que reconstruye el entorno completo desde GitHub en menos de diez minutos.

2. Diagnóstico

2.1 Por qué se pierden cambios

El sandbox donde se trabaja wave-app se reinicia periódicamente. Cuando eso ocurre, todo archivo que no haya sido commiteado y pusheado a GitHub se pierde. La especificación ya advierte esto en la regla 13.1: «todo lo local debe tener réplica en GitHub». La brecha no es de reglas, sino de ejecución: cuando se está depurando un bug difícil (por ejemplo, el ecualizador que la propia spec marca como no funcional), es común acumular varios archivos modificados sin commit durante horas, y si el sandbox se recicla en ese momento, el trabajo se evapora.

Una segunda causa de pérdida es el stash mal gestionado. Si se hace git stash para cambiar de tarea y luego se olvida, el stash queda huérfano y eventualmente se pierde al reciclar el sandbox. La tercera causa es el archivo nuevo sin trackear: git add solo agrega archivos ya conocidos; los untracked files quedan fuera del commit y se pierden.

El toolkit resuelve las tres causas con sync.sh, que hace add -A (incluye untracked), genera un mensaje de commit automático con timestamp y lista de archivos principales, y pushea en la misma operación. Para emergencias también permite stash con nombre descriptivo y recuperación posterior.

2.2 Por qué se acumulan errores

La causa raíz es la misma que la spec identifica en su regla 13.1: «compilar localmente antes de cada push». Cuando esa verificación se omite — porque el build local tarda minutos, porque se confía en que GitHub Actions lo detectará, o simplemente porque el desarrollador está apurado por subir un fix — el push llega con errores de compilación que ensucian el historial, rompen el APK que publican los Releases, y obligan a commits de reversión o a force-push no deseado.

Una segunda fuente de errores es la divergencia entre el sandbox y GitHub. Si el sandbox no hace pull antes de trabajar, los commits locales se construyen sobre una base vieja y al push se descubren conflictos que se resuelven apurados, introduciendo bugs. La tercera fuente es la reinstalación manual del toolchain cuando el sandbox se recicla: sin JDK 21 exacto, Gradle 8.14.3 exacto y SDK 36 con los build-tools correctos, el build local puede pasar y el de Actions fallar (o viceversa), generando falsos positivos y negativos.

El toolkit ataca las tres causas: safe-push.sh ejecuta ./gradlew assembleDebug antes del push y aborta si falla; el hook pre-push repite esa verificación de forma automática en cualquier git push a main; y bootstrap.sh instala el toolchain completo y versionado, idempotente, con espejos Aliyun para entornos con red restringida.

3. Principios del nuevo flujo

El flujo propuesto se construye sobre cinco principios que se aplican en todas las etapas y que conviene tener presentes antes de mirar los scripts concretos. Cada principio se mapea a una regla existente de la spec Wave, de modo que el toolkit no es una invención nueva sino la materialización operativa de lo que la spec ya exige.

3.1 GitHub es la única fuente de verdad

Todo lo que exista localmente — código, toolchain, scripts, configuración — debe tener réplica en GitHub. El sandbox se recicla y no se puede confiar en él para persistencia. Esto implica que cada cambio, por pequeño que sea, debe terminar en un commit pusheado antes de cerrar la sesión. El script sync.sh está diseñado exactamente para este principio: commitea lo que haya (incluso sin mensaje custom, lo genera automático) y pushea.

3.2 Build local antes de push, sin excepciones

Es la regla 13.1 textual de la spec. El hook pre-push la vuelve no opcional: cualquier intento de push a main dispara una compilación assembleDebug, y si esa compilación falla, el push se aborta. La única excepción legítima es una emergencia donde se necesita subir un fix crítico con un build roto conocido (por ejemplo, para que un colega lo descargue y depure); en ese caso, la variable de entorno WAVE_SKIP_BUILD_CHECK=1 permite saltarse el check, pero debe quedar registrado en el mensaje del commit por qué se hizo.

3.3 Toolchain reproducible y versionado

JDK 21 Temurin, Gradle 8.14.3, Android SDK 36 con build-tools 36.0.0, espejos Aliyun primero. Estas versiones son las que usa el workflow build.yml de Actions y deben ser las mismas del sandbox. bootstrap.sh las instala en $WAVE_TOOLCHAIN_DIR (por defecto $HOME/wave-toolchain) de forma idempotente: si ya están, no hace nada; si falta algo, lo instala; y deja un archivo env.sh que se sourcea para activar JAVA_HOME, GRADLE_HOME, ANDROID_HOME y PATH en un solo comando.

3.4 Sincronización frecuente vs. publicación cuidadosa

Hay dos operaciones distintas que se confunden a menudo: sincronizar (guardar cambios para no perderlos) y publicar (poner una versión confiable en main). sync.sh hace lo primero: commitea y pushea sin build, ideal para cerrar el momento de trabajo antes de irse. safe-push.sh hace lo segundo: build, commit solo si pasó, push, y actualización de CHANGELOG.md con la entrada correspondiente. Mezclar los dos (usar sync.sh para publicar) es uno de los anti-patrones a evitar.

3.5 Recuperación ante desastre como ciudadano de primera clase

Asumir que el sandbox se va a perder no es pesimismo, es realismo. recover.sh hace backup de cambios sin commitear a /tmp/wave-recover-<timestamp>.tar.gz, elimina wave-app/, re-clona desde GitHub, verifica el toolchain, instala los hooks y muestra el dashboard de status. Es el script al que se llama cuando nada más funciona. Su existencia como parte del toolkit significa que incluso el peor escenario (sandbox reciclado con trabajo sin commit) tiene un camino de retorno.

4. Setup inicial (bootstrap)

El setup inicial se ejecuta una sola vez por sandbox nuevo. Su objetivo es dejar el toolchain completo y el repo wave-app clonado, con los hooks instalados. Después de eso, el flujo diario es mucho más liviano (ver capítulo 5).

4.1 Pasos del setup

- `cd /home/z/my-project/scripts/wave-toolkit` (o donde se haya clonado Wave2)
- `chmod +x *.sh` — dar permisos de ejecución a todos los scripts
- `./bootstrap.sh` — instala JDK 21 + Gradle 8.14.3 + Android SDK 36 (idempotente, con espejos Aliyun)
- `source $WAVE_TOOLCHAIN_DIR/env.sh` — activa JAVA_HOME, GRADLE_HOME, ANDROID_HOME, PATH en el shell actual
- `export WAVE_GITHUB_TOKEN=ghp_xxx` — si el repo es privado, setear el PAT en el entorno

- `./workon.sh` — clona wave-app, pull de main, verifica estado, muestra dashboard
- `./install-hooks.sh` — instala el hook pre-push en wave-app/.git/hooks/

4.2 Verificación

Después del setup, el comando bootstrap.sh --check debería imprimir cuatro líneas «OK» (JDK, Gradle, Android platform, Android build-tools) y ninguna «ERR». Si alguna falta, volver a ejecutar bootstrap.sh sin argumentos reinstala lo que falte. El comando status.sh (ver capítulo 6) muestra el dashboard completo y permite confirmar que todo está en su lugar antes de empezar a editar código.

> **Importante:** el archivo env.sh generado por bootstrap.sh contiene rutas absolutas al toolchain de este sandbox. Si el sandbox se recicla, ese env.sh deja de ser válido. La regla es: cada vez que se llama a workon.sh, este verifica si el toolchain está completo y, si no, llama a bootstrap.sh antes de activar el entorno.

5. Flujo de sesión diaria

Una vez completado el setup inicial, el flujo diario se reduce a tres comandos. La meta es que nunca haya más de una hora de trabajo sin commitear, para que el daño máximo ante un reciclaje del sandbox sea acotado.

5.1 Las cinco etapas

Etapa	Comando	Frecuencia	Duración típica
1. Inicio	./workon.sh	Al iniciar sesión	10-30 segundos
2. Edición	(editor)	Continuo	Variable
3. Sincronizar	./sync.sh	Cada 30-60 min	5-10 segundos
4. Publicar	./safe-push.sh	Cuando hay cambio estable	3-8 minutos (build)
5. Cerrar sesión	./sync.sh	Al terminar	5-10 segundos

Tabla 1. Flujo diario de cinco etapas con wave-toolkit.

5.2 Detalle de cada etapa

Etapa 1 — Inicio. workon.sh verifica que el toolchain esté completo (si no, llama a bootstrap.sh), clona wave-app si no existe, hace fetch + pull --ff-only de main, verifica que el working tree esté limpio (si no, avisa sin sobre-escribir nada), activa el entorno (JAVA_HOME, GRADLE_HOME, ANDROID_HOME, PATH), e imprime un dashboard con el último commit, la cantidad de archivos modificados, si el hook pre-push está instalado, y qué APK hay compilado.

Etapa 2 — Edición. Sin tooling específico; se edita con el editor preferido dentro de /home/z/my-project/wave-app/. Si se va a probar en dispositivo, compilar con ./gradlew assembleDebug directamente o con safe-push.sh --build-only (que es lo mismo pero sin pushear). El APK resultante queda en app/build/outputs/apk/debug/app-debug.apk.

Etapa 3 — Sincronizar. sync.sh hace git add -A (incluye untracked), genera un mensaje de commit automático con timestamp y top-3 de archivos modificados, commitea y pushea. No hace build. Es la red de seguridad: si el sandbox se recicla cinco minutos después, el trabajo ya está en GitHub. La frecuencia recomendada es cada 30-60 minutos de trabajo continuo, o antes de cualquier cambio grande que pueda romper algo.

Etapa 4 — Publicar. safe-push.sh es el comando cuidadoso. Ejecuta ./gradlew assembleDebug con --no-daemon --max-workers=1 y heap limitado (2048m heap, 512m metaspace, igual que el workflow de Actions). Si el build falla, aborta: no commitea, no pushea, no toca CHANGELOG. Si pasa, commitea (con mensaje automático tipo «[safe-push] build OK v2.4.0 — <fecha>» o con -m custom), actualiza CHANGELOG.md con una entrada con timestamp, hash del commit, versión y duración del build, y pushea a origin/main.

Etapa 5 — Cerrar sesión. Antes de salir del sandbox, sync.sh una vez más para garantizar que nada quede sin subir. Aunque se haya hecho safe-push.sh hace rato, puede haber cambios residuales (un archivo guardado al final, un .gitignore ajustado). sync.sh es barato y no rompe nada.

5.3 Diagrama de decisión

Cuando se está trabajando y aparece la duda «¿commiteo ahora o sigo?», el árbol de decisión es el siguiente:

```
  ¿Hay cambios sin commitear hace más de 30 min?
    │ SÍ → ./sync.sh  (guarda rápido, sin build)
    │ NO → seguís trabajando
    ↓
  ¿El cambio actual es estable y compila?
    │ SÍ → ./safe-push.sh  (build + commit + push + CHANGELOG)
    │ NO → seguir editando, sync.sh cuando estés inseguro del sandbox
    ↓
  ¿Vas a cerrar la sesión?
    │ SÍ → ./sync.sh una última vez, después ./status.sh para
confirmar
    │ NO → seguir en bucle
```

6. Catálogo de scripts

Referencia rápida de los siete scripts del toolkit. Todos aceptan --help para ayuda inline y se ejecutan desde cualquier directorio siempre que WAVE_APP_DIR y WAVE_TOOLCHAIN_DIR estén configurados (workon.sh los setea automáticamente).

6.1 Tabla maestra

Script	Para qué sirve	Modifica
bootstrap.sh	Instala JDK 21 + Gradle 8.14.3 + Android SDK 36 (idempotente, espejos Aliyun)	$WAVE_TOOLCHAIN_DIR
workon.sh	Clona/pulla wave-app, activa entorno, muestra dashboard inicial	$WAVE_APP_DIR/.git
sync.sh	Commit automático + push rápido (sin build, para no perder cambios)	$WAVE_APP_DIR/.git, origin
safe-push.sh	Build local + commit + push + CHANGELOG.md (regla 13.1 spec Wave)	$WAVE_APP_DIR/.git, origin, CHANGELOG.md
install-hooks.sh	Instala hook pre-push en wave-app/.git/hooks/ (build verification)	$WAVE_APP_DIR/.git/hooks/pre-push
status.sh	Dashboard de salud: git + toolchain + APK + sugerencias	Solo lectura
recover.sh	Recuperación ante desastre: backup + re-clone + bootstrap + hooks	$WAVE_APP_DIR completo, /tmp/wave-recover-*.tar.gz

Tabla 2. Catálogo maestro de scripts wave-toolkit.

6.2 Variables de entorno configurables

Todos los scripts leen estas variables. Si no están seteadas, usan defaults razonables para el sandbox estándar /home/z/my-project.

Variable	Default	Para qué
WAVE_TOOLCHAIN_DIR	$HOME/wave-toolchain	Dónde se instala JDK/Gradle/SDK

Variable	Default	Para qué
WAVE_APP_DIR	/home/z/my-project/wave-app	Dónde se clona el repo wave-app
WAVE_GITHUB_TOKEN	(vacío)	PAT de GitHub para push en repo privado
WAVE_REPO	github.com/Danielbarrantesv/wave-app.git	URL override del repo
WAVE_SKIP_BUILD_CHECK	(vacío)	Si se setea en 1, el hook pre-push salta el build

Tabla 3. Variables de entorno del toolkit.

7. Git hooks y verificación pre-push

El hook pre-push es la pieza que vuelve no opcional la regla 13.1 de la spec Wave: «compilar localmente antes de cada push». Una vez instalado, cualquier intento de git push a main dispara una compilación assembleDebug; si esa compilación falla, el push se aborta y el desarrollador tiene que arreglar el build antes de poder subir nada.

7.1 Qué hace el hook exactamente

El hook está en wave-app/.git/hooks/pre-push y es un script bash que: (a) lee las refs que se intentan pushear y descarta el stdin estándar de pre-push, (b) verifica que la branch actual sea main (si no, salta el check para permitir pushes a branches auxiliares de trabajo), (c) localiza el toolkit buscando safe-push.sh en rutas conocidas, (d) ejecuta safe-push.sh --build-only (que solo compila, no commitea ni pushea), y (e) si el build pasa, permite el push; si falla, aborta con código de salida distinto de cero y git cancela el push.

7.2 Cuándo saltarse el hook (emergencias)

Hay escenarios legítimos donde se necesita subir código con un build roto conocido: por ejemplo, cuando se está depurando un bug difícil y se quiere que un colega descargue el código actual para ayudarte, o cuando se va a depurar algo en un dispositivo donde el build local no puede correr por restricciones del sandbox. En esos casos, la variable de entorno WAVE_SKIP_BUILD_CHECK=1 desactiva el check para esa operación:

```
WAVE_SKIP_BUILD_CHECK=1 git push origin main
```

> **Advertencia:** el uso de WAVE_SKIP_BUILD_CHECK debe quedar registrado en el mensaje del commit, por ejemplo: «[skip-build-check] WIP: depurando ecualizador, build roto conocido en

> EqualizerManager.init». Esto deja traza en el historial y permite al siguiente desarrollador saber que ese commit no pasó verificación.

7.3 Instalación y desinstalación

El hook se instala con install-hooks.sh, que escribe el script en wave-app/.git/hooks/pre-push, le da permiso de ejecución y deja un marcador «wave-toolkit-managed» en el archivo para que el script de uninstall lo reconozca. La desinstalación es con install-hooks.sh --uninstall. Si el archivo pre-push ya existe y no tiene el marcador, install-hooks.sh lo sobre-escribe — conviene revisar a mano si había un hook custom antes de instalar.

8. Protocolo de recuperación ante desastre

recover.sh es el script al que se llama cuando nada más funciona. Los escenarios típicos son: el sandbox se recicló y se perdió wave-app completo; el working tree quedó corrupto por un merge mal resuelto; se quiere empezar limpio desde GitHub porque los cambios locales no se entienden. En todos los casos, GitHub es la fuente de verdad y recover.sh reconstruye el estado local a partir de él.

8.1 Qué hace recover.sh paso a paso

Paso	Acción	Detalle
1	Confirmación interactiva	Pide que se escriba SI literal para confirmar; aborta en cualquier otra respuesta
2	Backup de cambios locales	Si hay cambios sin commitear, los empaqueta en /tmp/wave-recover-<timestamp>.tar.gz
3	Eliminar wave-app/	rm -rf completo del directorio del repo local
4	Re-clone desde GitHub	git clone de WAVE_REPO (con token si hace falta), reescribe origin sin token
5	Verificar toolchain	bootstrap.sh --check; si incompleto, lanza bootstrap.sh
6	Instalar hooks	install-hooks.sh deja pre-push en su lugar
7	Dashboard final	status.sh muestra el estado reconstruido

Tabla 4. Pasos del protocolo de recuperación.

8.2 Dry-run y modos alternativos

Para inspeccionar qué haría recover.sh sin ejecutar nada, se usa --dry-run: imprime cada paso sin tocar el filesystem. Si se quiere recuperar el repo pero mantener el toolchain ya instalado (porque el sandbox no se recicló, solo se rompió el repo), se usa --keep-toolchain. Si se está seguro de que no hay cambios locales que valgan la pena respaldar, --no-backup salta el paso 2 y es más rápido.

> **Invariantes:** recover.sh nunca hace force-push ni borra datos en GitHub. Solo reconstruye local. El backup en /tmp se mantiene incluso después de la recuperación, por si se discovere que se perdió algo importante; conviene revisarlo antes de cerrar la sesión.

9. Anti-patrones a evitar

Estos son los errores más comunes que vuelven a introducir los problemas que el toolkit trata de resolver. Conviene revisarlos cada vez que se sienta la tentación de saltarse el flujo.

9.1 Editar directo en GitHub web

Editar archivos directamente en github.com es tentador para un fix rápido, pero rompe el principio de «build local antes de push»: el cambio llega a main sin haber pasado por compile local ni por el hook pre-push, y muy probablemente rompe el build de Actions en la siguiente corrida. Si se hace, hay que seguirlo inmediatamente con un workon.sh + safe-push.sh local que verifique que lo editado a mano compila.

9.2 Usar sync.sh para publicar versiones estables

sync.sh no hace build. Si se usa para publicar un cambio que el desarrollador cree que compila pero no verificó, se incumple la regla 13.1 y se corre el riesgo de romper main. La distinción clave: sync.sh es para «no perder cambios», safe-push.sh es para «publicar confiado». Si el cambio es sustantivo (más de un archivo, toca código de producción, afecta PlaybackController o cualquier clase marcada como CRITICAL en la spec), usar safe-push.sh.

9.3 git push --force a main sin aviso

Un force-push a main reescribe el historial y rompe a cualquier otro contributor o cualquier clone existente del repo. Si se llega a necesitar (por ejemplo, para borrar un commit que subió información sensible), coordinar antes con cualquier otra persona que tenga clone, y dejar nota en el mensaje del commit que reemplaza. El hook pre-push no bloquea force-push, pero la regla de oro del capítulo 15.1 de la spec dice explícitamente: «nunca regenerar el keystore ni cambiar firma» — misma lógica aplica al historial.

9.4 WAVE_SKIP_BUILD_CHECK=1 como costumbre

La variable existe para emergencias, no para rutina. Si se encuentra uno seteándola en cada push, es señal de que el build está roto y no se está arreglando. La acción correcta es arreglar el build, no saltarse el check. Si el build falla consistentemente por un problema del toolchain (no del código), ejecutar bootstrap.sh --force para reinstalar limpio.

9.5 Acumular horas de trabajo sin commitear

Es el anti-patrón original que el toolkit combate. Aunque sync.sh exists, si no se llama, no hace nada. La disciplina personal de commitear cada 30-60 minutos es la única defensa real contra el reciclaje del sandbox. El dashboard de status.sh muestra la cantidad de archivos modificados como recordatorio visual: si ese número pasa de 10, es hora de ejecutar sync.sh.

9.6 Reinstalar el toolchain de cero sin consultar el respaldo

La regla 13.1 textual dice: «nunca instalar el toolchain de cero sin consultar el respaldo de GitHub». Esto significa: antes de instalar JDK/Gradle/SDK a mano, verificar qué versiones usa el workflow build.yml de Actions (que está en el repo, por lo tanto respaldado) y usar exactamente esas. bootstrap.sh está codificado con esas versiones exactas (JDK 21, Gradle 8.14.3, SDK 36, build-tools 36.0.0); si se modifica, debe ser con un commit que explique por qué.

10. Mapeo con reglas existentes de la spec Wave

El toolkit no inventa reglas nuevas; materializa las que ya están en los capítulos 13.1, 14 y 15.1 de la especificación técnica v2.4.0. Esta tabla hace explícito el mapeo, para que quede claro que cada script responde a una regla existente y no a una invención ad-hoc.

Regla spec Wave	Capítulo	Script que la materializa
Compilar localmente antes de cada push	13.1	safe-push.sh + hook pre-push
Todo lo local debe tener réplica en GitHub	13.1	sync.sh (commit + push frecuentes)
Nunca instalar toolchain de cero sin consultar respaldo	13.1	bootstrap.sh (versiones fijas en script)
Flujo: cambio, build, commit, push, verificar Actions, APK	13.1	safe-push.sh + status.sh
Token de push con scopes repo + workflow	13.1	WAVE_GITHUB_TOKEN env var
Respetar invariantes CRITICAL	14	safe-push.sh aborta si build (que toca CRITICAL) falla

Regla spec Wave	Capítulo	Script que la materializa
toggleFavorite y mutaciones nunca vacían DB	14.1	build verification cubre regressiones
keepScreenOn / keepAppOpen defaults TRUE	14.1	build verification detecta si se rompen
Lista Favoritos no se renombra ni elimina	14.1	build verification detecta si se rompe
Deduplicación obligatoria en importación y restauración	14.1	build verification cubre tests de regressión
Recorrido de carpetas limita profundidad a 8	14.1	build verification cubre tests
Compilar local antes de push; un cambio, un push, verificar Actions, instalar APK del Release	15.1	Flujo completo caps. 5 y 6
Todo lo local con réplica en GitHub: el sandbox se recicla, GitHub es fuente de verdad	15.1	sync.sh + recover.sh
Nunca regenerar keystore ni cambiar firma si pasa a release	15.1	(convención de proyecto, no script; el keystore está en repo)
Respetar invariantes CRITICAL del cap. 14	15.1	safe-push.sh aborta build que los rompa
Nombre visible Wave y acento #32C98B	15.1	(convención de diseño, no script)

Tabla 5. Mapeo entre reglas de la spec Wave v2.4.0 y scripts del toolkit.

Apéndice A — Comandos rápidos

Referencia de bolsillo para tener a mano durante el trabajo diario. Asume que se está en /home/z/my-project/wave-app con el entorno activado (source $WAVE_TOOLCHAIN_DIR/env.sh).

A.1 Inicio de sesión

Comando	Qué hace	Cuándo usarlo
./workon.sh	Clona o actualiza wave-app, activa entorno, dashboard	Al abrir el sandbox

Comando	Qué hace	Cuándo usarlo
./bootstrap.sh --check	Verifica toolchain sin instalar nada	Cuando algo falla y no sabes qué
source $WAVE_TOOLCHAIN_DIR/env.sh	Activa JAVA_HOME, GRADLE_HOME, ANDROID_HOME, PATH	En cada shell nuevo

A.2 Trabajo

Comando	Qué hace	Cuándo usarlo
./gradlew assembleDebug	Compila APK debug (sin push)	Probar en dispositivo
./safe-push.sh --build-only	Idem pero vía toolkit (mismo resultado)	Probar con flags del toolkit
./status.sh	Dashboard git + toolchain + APK	Cuando dudas del estado
./status.sh --json	Mismo dashboard en JSON	Integración con otros tools

A.3 Sincronización y publicación

Comando	Qué hace	Cuándo usarlo
./sync.sh	Commit auto + push (sin build)	Guardar rápido
./sync.sh -m "mensaje"	Commit con mensaje custom + push	Guardar con contexto
./sync.sh --no-push	Commit sin push (lo dejas local)	Cuando no quieres subir aún
./sync.sh --stash nombre	Stash con nombre descriptivo	Cambiar de tarea sin perder
./safe-push.sh	Build + commit auto + push + CHANGELOG	Publicar versión estable
./safe-push.sh -m "fix: eq init"	Build + commit custom + push	Publicar fix específico

A.4 Emergencias

Comando	Qué hace	Cuándo usarlo
WAVE_SKIP_BUILD_CHECK=1 git push origin main	Push saltando build check (emergencia)	Build roto conocido, need subir ya
./recover.sh	Re-clone + bootstrap + hooks desde GitHub	Sandbox reciclado / repo corrupto
./recover.sh --dry-run	Muestra qué haría recover sin ejecutar	Inspeccionar antes de romper
./recover.sh --no-backup	Recuperar sin backup de cambios locales	Seguro que nada que respaldar
./install-hooks.sh --uninstall	Quita el hook pre-push	Hook impide push que necesitas

Apéndice B — Estructura de archivos del toolkit

El toolkit vive en Wave2/scripts/wave-toolkit/ y se mantiene en el repo Wave2 (github.com/Danielbarrantesv/Wave2) para que sobreviva al reciclaje del sandbox. La estructura es:

```
scripts/wave-toolkit/
├── README.md            — Documentacion rapida del toolkit
├── bootstrap.sh         — Instalacion del toolchain (JDK + Gradle + Android SDK)
├── workon.sh            — Inicio de sesion: clone/pull + env + dashboard
├── sync.sh              — Auto-commit + push rapido
├── safe-push.sh         — Build + commit + push + CHANGELOG (regla 13.1)
├── install-hooks.sh     — Instala hook pre-push en wave-app/.git/hooks/
├── status.sh            — Dashboard de salud del proyecto
├── recover.sh           — Recuperacion ante desastre
└── git-hooks/
    └── pre-push         — Plantilla del hook (instalada por install-hooks.sh)
```

El repositorio Wave2 se clona al iniciar cualquier sandbox nuevo y, con él, el toolkit completo. El primer comando después de un reciclaje es, por tanto, clonar Wave2 y ejecutar workon.sh; este último se encarga del resto (incluyendo clonar wave-app si falta).

> **Convención:** los scripts del toolkit nunca se editan dentro de wave-app/. Si se necesita modificarlos, se hace en Wave2/scripts/wave-toolkit/, se commitea y pushea a Wave2, y se

> sincroniza al sandbox del wave-app con un git pull de Wave2. Esto mantiene la separación: Wave2 contiene el tooling, wave-app contiene el producto.